GEMMA 4 DEVELOPER AGENT COMPETITION · PAPER TRACK · NOVEMBER 2026

Can a small local agent redo a cloud agent's work?

A cloud AI agent built a health-data code project. We turned 93 of its real changes into test tasks and asked a small model, Gemma 4 E4B, to make them again.

Health Transformation Lab · **Evidence you can check** SYNTHETIC PATIENT RECORDS

John Bowyer · personal project, not affiliated with any organization

WHO IT'S FOR, WHAT WE ASK, WHAT CHANGES

Built for people who build and test AI coding agents

Who	What we ask them to do	What changes
Test builders who turn a code project's history into tasks	Check every task: does its description say what its tests look for?	Scores are reported on tasks that can tell one agent from another
Agent builders working with small models that run on one machine	Run their agent on the 93 tasks, with the competition's scoring program	A score they can compare, on health-data code as well as popular libraries
Teams whose code is written with AI agents	Keep a written request, a proposed change and a review for every change	Their own history can become a set of test tasks

The impact: a set of tasks that says which of its tasks can measure an agent. So far it is one code project and one model, and an AI did the check of the descriptions, with a person checking part of it.

A personal project: no partner has been contacted

THE PROBLEM

Tests for AI coding agents use a few popular libraries. Health-data code fails in a different way.

- The best-known test, SWE-bench, draws on twelve popular Python libraries. The competition's development set draws on four.
- In a **data project** the costly mistake is silent: the pipeline finishes and a number in its table is wrong.
- A **project built by an AI agent** can leave an unusually complete record: an issue, a pull request and a review for each change.

960
saved changes (commits) in the project

775
of those 960 are marked as written by the AI agent

2,269
automatic tests, run on every pull request

240
pull requests, each with a written review kept in the project

One person working with a cloud AI coding agent built this project in fifteen days, without typing code. **What can a small model on one machine take out of that loop?**

Source: the paper, Sections 1 and 3

WHAT WAS BUILT

197 real code changes became 93 test tasks, each checked by the competition's own scoring program



The scoring works. Given the real fix, the scoring program (the evaluator) passes 93 of 93 tasks. Given no change, it passes 0 of 93.

Anyone can run it. Our open test harness gave the scoring program's verdict on every answer compared: 25 of 25 and 23 of 23.

Each task comes with a copy of the code, a map of the code (a graph), and where its description came from: an issue, a pull request or a commit message.

Source: the paper, Sections 4 and 5

WHAT THE SMALL MODEL DID

On its own, the small model fixed 5 of 48 tasks. Extra help did not clearly raise that.

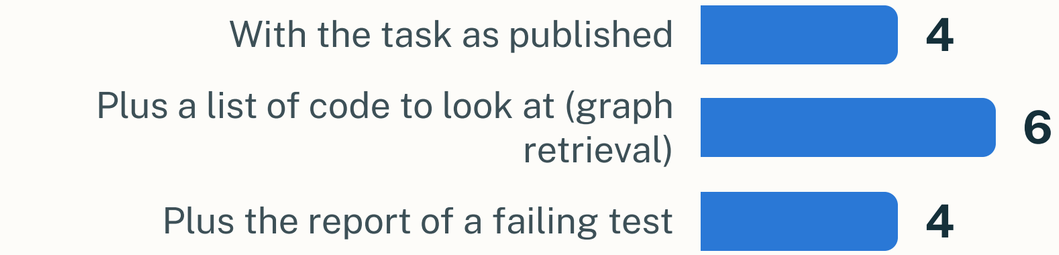
Tasks fixed working alone, on free T4 GPUs

5 of 48

Gemma 4 E4B, taking the cheapest tasks first (10.4%, 95% CI 4.5–22.2). All five were small: a CI rule, a warning message, a label parser and two settings.

3 of 23 of the competition's own development tasks fixed, under its scoring program.

Tasks fixed, of all 93, in an experiment planned in advance (one A100 GPU)



No difference detected. Gaps this small could be chance (exact McNemar p = 0.5 and 1.0). A second, unplanned session on an L4 GPU fixed 5 and 8.

The failing test's report comes from the hidden test, so it is not something a benchmark would give.

Source: the paper, Section 6. "Fixed" is the paper's "resolved": the task's hidden tests pass

THE MAIN FINDING

The model fixed 8 of the 17 clearly described tasks, and only 1 of the other 76

"Clearly described" means the task's description says what its tests check. An AI made that check on all 93 tasks, and a person checked 42 of its verdicts.

Clearly described tasks fixed at least once

8 of 17

Counting every attempt in the five Colab runs: 26 of 85

Other tasks fixed at least once

1 of 76

Counting every attempt: 1 of 380. Their descriptions leave out something the tests need.

Being pointed at the right code was not enough. In 41 tasks the search listed a function the fix changes. Of the 7 clearly described ones, 5 were fixed in some run. Of the other 34, none.

DESCRIBED, NOT TESTED

Counts of what happened, with no statistical test; the second comparison was made after the runs. The paper calls the 17 the core split (Section 6).

WHAT IT COST

A fix took 1.8 to 1.9 GPU-hours. Most of a task's time was the model's own calls.

Share of a task's time spent in the model's own calls

66–83%

Another 14–15% went to summarising older turns (context compaction). From the Kaggle runs' timing records (OpenTelemetry spans). 45 of 207 runs ended at the model's context window.

GPU-hours for each fix, on four L4 GPUs

1.9 and 1.8

1.9 with graph retrieval and 1.8 with the test report: about 1.59 and 1.50 US dollars at a median rental of 0.83 an L4-hour. Kaggle's GPUs were free.

Hours of agent time for each fix, on the T4 baseline

0.38 vs 1.80

0.38 on the 13 clearly described tasks it ran, against 1.80 over all 48. Those 13 took 17% of its 9.0 agent-hours and gave 4 of its 5 fixes.

What it means: the time goes to the model's own calls, not to the graph tools, and an hour spent on clearly described tasks bought the most fixes.

Source: the paper, Section 6, "Time, GPUs and cost". The prices are rental estimates, not money spent

WHAT TO DO WITH IT

Use the 93 tasks. Report scores on the 17 clearly described ones.

Run your agent on them. 93 tasks, free to use (CC BY 4.0), in the competition's layout: its scoring program runs on them as published.

Report scores on the 17. Over every run, 9 of the 10 different tasks this model has fixed are among them (the paper's core split).

Check your own tasks. Ask of each one whether its description says what its tests check, before you report a score on it.

Check any number. Each one in the paper has a row in `paper/sources.md` naming the file it came from.

Tasks: kaggle.com/datasets/jcblearning/health-transformation-lab-coding-agent-tasks
Paper, sources and agent configuration: github.com/healthtransformationlab/gemma-developer-agent

Drill-down

Backup slides for questions. The full detail is in the paper.

- What this does not show
- How our tasks compare with the competition's own
- How you can check every number

DRILL-DOWN · LIMITS

What this does not show

Only one new code project, set beside the competition's development set.

No comparison with the cloud agent. The agent that made these changes, with a person in the loop, was not rerun on the task descriptions alone.

A small experiment. With 4 of 93 tasks fixed as the starting point, only a large effect could show.

Not a leaderboard score. Every run under the competition's scoring program had 5 minutes and 40 tool calls a task, not the 60 and 100 allowed.

An AI checked the descriptions. A person checked 42 of its verdicts, the 17 clearly described tasks among them, in summary and after seeing the AI's; 51 have AI ratings only.

The model was told something was wrong. Each task comes with a failing test: noticing a silent defect unprompted is not measured.

Read the results as: what one small model did on one project's tasks, with the AI's verdicts checked by one person in part.

Source: the paper, Section 7

DRILL-DOWN · THE COMPETITION'S OWN TASKS

In 76 of our 93 tasks the tests need something the description never gives. In the competition's: 14 of 40.

How many tasks…	Our tasks	The competition's development tasks
have a fix that changes code the code map (graph) holds	51 of 93	114 of 129
of those: can be reached by a name or a call link; with similarity search too	22; 42	52; 66
have a description that is vague or worse (AI check)	19 of 93	11 of 40
have tests that need a name or text the description never gives	**76 of 93**	**14 of 40**
were fixed with the task as published, on Kaggle (ours: 21 tasks ran)	0 of 21	3 of 23

The two sets differ in their tests more than in their descriptions. The check and the agent took a seeded 40 of the competition's 129 development tasks; a Kaggle notebook can verify 23 of them.

Source: the paper, Section 6, "Beside the competition's own tasks"

DRILL-DOWN · EVIDENCE YOU CAN CHECK

How you can check every number

A source for every number. `paper/sources.md` has a row for each figure in the paper, naming the file it came from, and keeps the rows of text that was cut.

A plan before the results. The experiment's plan was fixed and fingerprinted (hashed) before any result, and each addition to it before its run started (source repository, issues 674 and 687).

Checked on every change. One script checks the paper against the competition's rules, and another checks that every number in this deck is one the paper has.

What went wrong is in the paper. On Colab the network was open to the agent: 3 of the five runs' 687 commands ran an installer, in tasks no run fixed, and none left its workspace.

Not a result: anything from a dry run or a sample is labelled as one, and the unplanned second session is described beside the planned experiment, never added to it.

Sources: paper/sources.md, paper/rules_check.md and the paper, Section 6